Unit 1: Introduction to Software Development

Complete exercise solutions — MCQs, Short Questions, Long Questions

EXERCISE SOLUTIONS — Unit 1: Introduction to Software Development

Multiple Choice Questions

1. Primary purpose of the Software Development Life Cycle (SDLC) is to:

• a) design websites

• b) deliver high-quality software within time and cost estimates

• c) manage database systems

• d) create hardware components

Answer: b) deliver high-quality software within time and cost estimates

2. A type of requirement specifying system performance is:

• a) Functional Requirements

• b) Non-Functional Requirements

• c) Technical Requirements

• d) Operational Requirements

Answer: b) Non-Functional Requirements

3. Role of a framework in the context of SDLC is to:

• a) write code from scratch

• b) provide a structured foundation with predefined components and architectures

• c) manage hardware

• d) perform manual testing

Answer: b) provide a structured foundation with predefined components and architectures

4. Software development model involving short cycles or sprints is:

• a) Waterfall Model

• b) Agile Methodology

• c) Lean Software Development

• d) Scrum

Answer: b) Agile Methodology

5. Crucial aspect of comprehensive project planning is:

• a) Understanding the project scope and tasks

• b) Deciding the project's colour scheme

• c) Hiring a large development team

studysapiens.com

• d) Ignoring potential risks

Answer: a) Understanding the project scope and tasks

6. Factor that does NOT influence the cost estimation of a software project:

• a) Scope of the project

• b) Technology stack

• c) Number of meetings held

• d) Operational costs

Answer: c) Number of meetings held

7. The purpose of Use Case Diagrams is to:

• a) document the system's architecture

• b) identify and document the system's functional requirements

• c) illustrate the database schema

• d) define the system's user interface design

Answer: b) identify and document the system's functional requirements

Short Questions

1. Differentiate between functional and non-functional requirements.

Functional requirements describe what the system should do — the basic features and functions. For example, in a Library Management System, allowing user registration and book borrowing are functional requirements. Non-functional requirements describe how well the system performs — qualities like speed, reliability, and security. For example, handling 1000 simultaneous users or maintaining 99.9% uptime are non-functional requirements.

2. Explain why the testing phase is important in SDLC, and provide two reasons for its significance.

The testing phase is important because: (1) It ensures the software meets the requirements — testing verifies that the product works as intended and matches what the customer asked for. (2) It finds and fixes errors early — defects discovered during testing are much cheaper to fix than those found after deployment. Testing also builds confidence in the product's quality and reliability.

3. Illustrate the concept of continuous integration in Agile Methodology and discuss its importance in software development.

Continuous integration (CI) means developers frequently merge their code changes into a shared repository, and automated builds and tests run each time. For example, when a developer pushes new code, the system automatically builds the software and runs tests. Its importance: it detects integration errors early, improves code quality, reduces the risk of large failures, and allows teams to deliver working software more frequently and reliably.

4. Evaluate the main steps involved in risk assessment and management, and assess their importance in a software project.

The main steps are: (1) Risk identification — listing possible risks such as delays, budget overruns, or technical failures. (2) Risk analysis — estimating the probability and impact of each risk. (3) Risk response planning — deciding how to avoid, reduce, transfer, or accept each risk. (4) Monitoring — tracking risks throughout the project. Their importance: managing risks prevents costly surprises, keeps the project on schedule and budget, and improves the chances of successful delivery.

5. Explain the purpose of a Use Case Diagram in software development.

A Use Case Diagram visually represents system functionality from the user's perspective. It shows the actors (users or external systems) and the use cases (tasks the system must perform). Its purpose is to capture functional requirements clearly, define the system's boundaries, and help developers and stakeholders agree on what the system should do. For example, in an online shopping platform, actors include Customer, Administrator, and Delivery Personnel, with use cases like Browse Products and Process Orders.

6. Compare and contrast a Sequence Diagram with an Activity Diagram, highlighting the key differences.

A Sequence Diagram shows the order of interactions between objects over time — it focuses on which object sends messages to which, and in what sequence. An Activity Diagram shows the flow of activities or workflow — it focuses on the sequence of tasks, decisions, and branching. Key differences: Sequence Diagrams emphasise object interaction and message passing, while Activity Diagrams emphasise control flow and business processes. Sequence Diagrams are best for detailed system interactions; Activity Diagrams are best for workflows and algorithms.

7. Describe the Factory Pattern and explain how it differs from directly creating objects, with an example.

The Factory Pattern is a design pattern that provides an interface for creating objects without specifying their exact class. Instead of calling a constructor directly (new Car()), the client asks a factory to create the object. This differs from direct creation because the client does not need to know the concrete class — the factory decides which class to instantiate. Example: a VehicleFactory.createVehicle(type) method returns a Car or Truck depending on the type, so the client code stays flexible and decoupled from specific classes.

Long Questions

1. Design a flowchart for a user registration process in a software application. Outline its key steps.

Key steps of the flowchart: (1) Start — user opens the registration page. (2) Enter username — check if the username is available. (3) Enter email — validate the email format. (4) Enter password — check password strength and confirmation match. (5) Enter personal details — name, date of birth, phone number. (6) Accept terms and conditions — user must agree. (7) Submit the form. (8) Validate all inputs — if any validation fails, return to the relevant step. (9) Save the user record to the database. (10) Send a confirmation email. (11) End — display a success message. The flowchart uses standard symbols: oval for start/end, parallelogram for input/output, rectangle for process, diamond for decision.

2. Imagine you are managing a project to develop a simple mobile application. Describe how you would use the Agile Methodology to handle this project.

I would use Agile as follows: (1) Product Backlog — I would create a prioritised list of features (login, profile, main screen, settings). (2) Sprint Planning — I would break the work into short sprints of 1–2 weeks, selecting the most important features for each sprint. (3) Daily Stand-ups — the team meets briefly each day to share progress and blockers. (4) Development with CI and TDD — code is integrated continuously and tested automatically. (5) Sprint Review — at the end of each sprint, I would demo the working app to stakeholders and gather feedback. (6) Retrospective — the team discusses what went well and what to improve. This iterative approach allows flexibility, early feedback, and steady delivery of working software.

3. Consider an online banking system. Create a Use Case Diagram to show the interactions between customers, bank staff, and the system.

Actors: Customer, Bank Staff (Teller/Manager), and the Banking System. Use cases for the Customer: Register Account, Login, View Balance, Transfer Funds, Pay Bills, Download Statement. Use cases for Bank Staff: Verify Identity, Open Account, Process Transactions, Resolve Complaints, Generate Reports. The Banking System interacts with both actors — for example, the Customer actor connects to 'Transfer Funds', and the Bank Staff actor connects to 'Process Transactions'. The system boundary encloses all use cases. This diagram clearly shows who can do what and how the system serves each actor.

4. You are developing a food delivery application. Create a Sequence Diagram to show the process of placing an order, from the customer selecting items to the delivery of the order.

Objects: Customer, Mobile App, Restaurant, Payment Gateway, Delivery Rider. Sequence: (1) Customer selects items and sends the order to the App. (2) App sends the order to the Restaurant. (3) Restaurant confirms availability and sends the bill to the App. (4) App sends the bill to the Customer. (5) Customer pays via the Payment Gateway. (6) Payment Gateway confirms payment to the App. (7) App confirms the order to the Restaurant. (8) Restaurant prepares the food and notifies the App. (9) App assigns a Delivery Rider. (10) Rider collects the food and delivers it to the Customer. (11) Customer receives the order and the App sends a confirmation. Each arrow represents a message between objects in chronological order.

5. Discuss the importance of software development tools in the software development process. Explain the role of language editors, translators, and debuggers in creating and maintaining software. Provide examples of each tool and describe how they contribute to the efficiency and accuracy of software development.

Software development tools increase efficiency, accuracy, and quality. Language editors (e.g., VS Code, PyCharm) provide syntax highlighting, auto-completion, and code organisation, helping developers write correct code faster. Translators/compilers (e.g., Python interpreter, GCC) convert human-readable source code into machine-executable code; interpreters run code line by line while compilers translate the whole program first. Debuggers (e.g., GDB, pdb) allow developers to run code step by step, inspect variables, and find the exact location of errors. Together, these tools reduce development time, minimise bugs, and improve the reliability of the final software.

studysapiens.com